Project Design Phase-II
Technology Stack (Architecture & Stack)

Date	19 Feb 2026
Team ID	LTVIP2026TMIDS83753
Project Name	Weather-Based Prediction of Wind Turbine Energy Output: A Next-Generation Approach to Renewable Energy Management

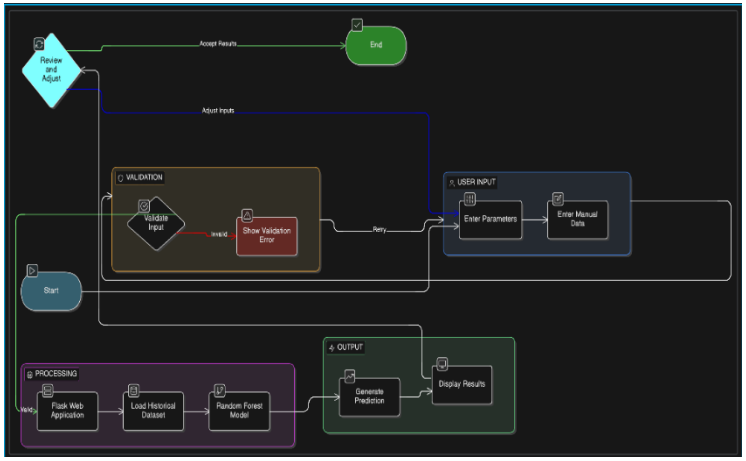
Technical Architecture:

The Weather-Based Prediction of Wind Turbine Energy Output system follows a layered architecture that integrates a web-based user interface with a backend machine learning model. The architecture enables real-time prediction of wind energy output based on weather parameters provided by the user.

The system is designed to ensure scalability, maintainability, and real-time processing capability.

Example: AI-Powered Backend System for Real-Time Wind Energy Prediction

Reference: https://developer.ibm.com/patterns/ai-powered-backend-system-for-order-processing-during-pandemics/



Guidelines:

The system should be designed using a layered architectural approach consisting of:

- Presentation Layer
- Application Layer
- Machine Learning Layer

The architecture should support real-time data processing and avoid batch execution to ensure immediate prediction results upon user input.

Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1.	User	Wind farm operator accessing the system	Web Browser
2.	Web Dashboard	User interface for input and result display	HTML, CSS
3.	Input Form	Collects wind speed and theoretical power	HTML Form
4.	Web Server	Handles HTTP requests and routing	Flask (Python)
5.	Backend Logic	Processes input and calls ML model	Python
6.	Prediction Engine	Executes prediction logic	Scikit-learn.
7.	Machine Learning Model	Predicts wind energy output	Random Forest Regression
8.	Dataset	Historical wind turbine dataset	CSV File (T1.csv)
9.	Runtime Environment	Runs the application locally	Local Flask Server (127.0.0.1:5000)

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1.	Architecture Style	The system follows a layered architecture including Presentation, Application, and Machine Learning layers.	HTML, CSS, Flask, Scikit-learn
2.	Processing Mode	The system performs real-time prediction immediately after user input without batch processing.	Flask, Python
3.	User Interaction	Users can enter input values, view prediction results, and modify inputs for recalculation.	HTML Forms, Flask
4.	Input Handling	The system accepts manual data inputs such as wind speed and theoretical power.	HTML Forms, Python
5.	Prediction Engine	The application uses a trained machine learning model to generate energy output predictions.	Random Forest (Scikit-learn)
6.	Data Management	Historical wind turbine data is used to train the prediction model.	CSV File (T1.csv), Python
7.	Deployment Model	The application runs locally on a development server.	Flask Local Server (127.0.0.1:5000)
8.	Error Handling	Basic validation ensures that required fields are filled and values are numeric.	HTML Validation, Python
9.	Scalability	The system can be extended to cloud deployment and advanced ML models.	Flask, Cloud Platforms (Future)
10.	Extensibility	Future integration of weather APIs and LSTM models is possible.	Future integration of weather APIs and LSTM models is possible.

References:

https://c4model.com/

https://developer.ibm.com/patterns/online-order-processing-system-during-pandemic

https://www.ibm.com/cloud/architecture

https://aws.amazon.com/architecture

https://medium.com/the-internal-startup/how-to-draw-useful-technical-architecture-diagrams-2d20c9fda90d